

Stage 3: Reproduction and Implementation

Implementation of Graph Partitioning Algorithms and Reproduction Results

Team Members: Name 1, Name 2, Name 3

CS240: Algorithm Design and Analysis, Spring 2026

1 Implementation Overview

The implementation is organized as a small Python project. The code is placed under `src/`, with separate modules for graph generation, algorithms, metrics, simulation, experiment execution, and plotting. This modular organization makes it easier to test each component independently and to compare several graph partitioning methods under the same evaluation pipeline.

The main files are:

- `src/graphs.py`: benchmark graph construction;
- `src/algorithms.py`: greedy, spectral, and Kernighan–Lin partitioning algorithms;
- `src/metrics.py`: partition quality metrics;
- `src/simulation.py`: consensus averaging simulation;
- `src/experiments.py`: experiment runner and CSV export;
- `src/plotting.py`: generation of result figures.

The project uses Python with NumPy, NetworkX, pandas, matplotlib, and pytest. The implementation is deterministic for the reported experiments by using a fixed random seed.

2 Implemented Algorithms

2.1 Greedy Balanced Partitioning

The greedy baseline assigns vertices to partitions one at a time. Vertices are processed in decreasing weighted degree order so that high-impact vertices are placed early. For each vertex, the algorithm considers the partitions that have not reached their capacity. It chooses the feasible partition that gives the lowest incremental cut cost with respect to already assigned neighbors. The capacity is set to approximately $\lceil n/k \rceil$, so the method maintains near-perfect balance.

This baseline is intentionally simple. It provides a fast comparison point and helps show whether more sophisticated algorithms actually improve partition quality.

2.2 Recursive Spectral Bisection

The spectral method recursively bisects the largest current group until k groups are obtained. For a subgraph, it constructs the weighted adjacency matrix A , the degree matrix D , and the graph Laplacian $L = D - A$. It then computes the eigenvectors of L and uses the Fiedler vector to sort vertices. The sorted list is split at the midpoint.

This method is useful because it gives a strong initial partition for structured graphs. In grid and geometric graphs, the Fiedler vector often captures spatial structure, so spectral bisection can produce low-cut balanced partitions.

2.3 Kernighan–Lin k -Way Refinement

The implemented Kernighan–Lin method starts from a spectral partition. It then performs several global refinement passes. During each pass, the method loops over every pair of parts. For a pair (a, b) , it extracts the subgraph induced by vertices currently assigned to either a or b . NetworkX’s robust Kernighan–Lin bisection routine is applied to this two-part subproblem using the current assignment as the initial partition.

The candidate update is accepted only if it does not increase the global cut-weight objective. This acceptance rule makes the refinement monotone with respect to the measured cut value. The refinement stops early if a full global pass fails to improve the objective.

3 Reproduction Scope

The original Kernighan–Lin paper focuses on graph bisection. Our project reproduces the core algorithmic idea rather than the exact historical experimental setup. The reproduced components are:

1. local-search refinement based on swapping vertices between two sides;
2. repeated passes until no meaningful cut improvement is found;
3. use of the method as a refinement procedure rather than a standalone random partitioner;
4. empirical measurement of cut improvement and runtime cost.

The main adaptation is the k -way setting. Instead of one global bisection, we apply pairwise Kernighan–Lin refinement to pairs of parts inside a k -way partition. This is a practical adaptation for distributed optimization because the target system has k workers, not only two processors.

4 Benchmark Instances

The implementation evaluates the algorithms on four graph families:

- **Grid graphs:** structured graphs with strong local geometry;
- **Erdos–Renyi random graphs:** unstructured random graphs;
- **Random geometric graphs:** spatial graphs with local connectivity;

- **Zachary karate club graph:** a small real social network.

The benchmark suite contains eight instances:

Graph	Size Parameter	$ V $	$ E $	k
grid	6	36	60	4
grid	8	64	112	4
grid	10	100	180	4
erdos_renyi	40	40	93	4
erdos_renyi	80	80	248	4
geometric	50	50	330	4
geometric	90	90	628	4
karate	34	34	78	2

5 Metrics

The primary metric is cut weight:

$$\text{cut}(p) = \sum_{(u,v) \in E, p(u) \neq p(v)} w_{uv}.$$

Lower cut weight means fewer or cheaper cross-worker dependencies.

The second metric is normalized cut, which divides boundary weight by the volume of each part and sums the ratios. This helps evaluate whether the cut is small relative to the connectivity of each partition.

The third metric is balance ratio:

$$\rho(p) = \frac{\max_i |\{v : p(v) = i\}|}{|V|/k}.$$

Values close to 1 indicate balanced workload.

The fourth metric is runtime in seconds. This measures the computational cost of each partitioning algorithm.

Finally, the consensus simulation reports final consensus error and the number of cross-partition messages. The simulation keeps the original communication graph fixed, so partitions do not change the convergence dynamics; instead, they change how many messages cross worker boundaries.

6 Reproduced Results

The experiments reproduce the expected qualitative behavior of Kernighan–Lin-style refinement. Across the benchmark suite, the average cut weights are:

$$\text{greedy} = 98.25, \quad \text{spectral} = 72.75, \quad \text{Kernighan–Lin} = 66.88.$$

The average cross-partition message counts in the consensus simulation are:

$$\text{greedy} = 15460, \quad \text{spectral} = 11360, \quad \text{Kernighan–Lin} = 10440.$$

The Kernighan–Lin refinement achieves the lowest average cut and the lowest average cross-worker traffic. It gives the best cut on six of the eight graph instances. This supports the intended reproduction target: local refinement often improves partition quality compared with a simpler baseline and can also improve a spectral initialization.

7 Deviation and Failure Analysis

There are two important deviations from the original paper. First, the original Kernighan–Lin method is a two-way graph partitioning algorithm, while our application requires k parts. We therefore use pairwise two-way refinement as a component inside a k -way framework.

Second, the project uses NetworkX’s implementation of pairwise Kernighan–Lin bisection rather than reimplementing every low-level detail from scratch. This choice improves robustness and lets the project focus on algorithmic comparison, adaptation, and evaluation. The core reproduced algorithmic behavior remains the local-search cut refinement.

The consensus experiment also has a limitation: it measures cross-worker traffic but does not alter the underlying communication topology. Therefore, different partitions have nearly identical consensus error on the same graph. This is not a failure of the partitioning algorithms; it reflects the design of the simulation. The simulation isolates communication placement cost rather than changing the numerical averaging process.

8 How to Run

The experiments can be reproduced with:

```
python src/experiments.py --output-dir outputs --seed 7
```

This command writes the raw CSV file to `outputs/partition_results.csv` and saves figures under `outputs/figures/`. Unit tests can be run with:

```
python -m pytest
```